💻 DAY 2: MASTER HANDS-ON LAB & COMMANDS RUNBOOK

OpenMP Multithreading, MPI Distributed Communication & Intel oneDAL Machine Learning Acceleration

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 2 Hands-On Server Lab (01:30 PM – 04:30 PM | 3 Hours, Part of 6-Hour Day 2 Masterclass)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

📌 Executive Architecture & Lab Pipeline

```
+-------------------------------------------------------------------------------+
|                            DAY 2 HANDS-ON COMMAND PIPELINE                    |
+-------------------------------------------------------------------------------+
| STAGE 0: Intel AI Analytics Environment Setup & Toolkit Verification          |
| STAGE 1: C++ OpenMP Multithreading Benchmark & Core Scaling (1, 2, 4, 8 Threads)|
| STAGE 2: OpenMP Data Decomposition & False Sharing Performance Profiling      |
| STAGE 3: C++ MPI Distributed Memory Message Passing (Point-to-Point & Collective Reduction)|
| STAGE 4: Intel-Optimized NumPy (oneMKL) & High-Performance Data Processing    |
| STAGE 5: Intel Extension for Scikit-learn (`sklearnex`) 1M Sample Clustering Benchmark|
| STAGE 6: End-to-End Supervised ML Acceleration: Random Forest & Support Vector Machines (SVM)|
+-------------------------------------------------------------------------------+
```

🚀 STAGE 0: Environment Setup & Toolkit Verification

Verify active Python environment and presence of Intel AI Analytics Toolkit components:

```
# 1. Verify Python and GCC/Intel compilers
python3 --version
g++ --version
which icpx 2>/dev/null || echo "icpx not installed, using g++"

# 2. Install Intel Extension for Scikit-learn and benchmarking datasets (Takes ~10 seconds)
pip install -q scikit-learn-intelex scikit-learn numpy pandas matplotlib

# 3. Verify sklearnex dynamic patch status
python3 -c "
import sklearnex
sklearnex.patch_sklearn()
import sklearn.cluster
print('[+] sklearnex patch successful! Using accelerated backend:', sklearn.cluster.KMeans.__module__)
"
```

⚡ STAGE 1: C++ OpenMP Multithreading & Core Scaling Benchmark

Writes, compiles, and benchmarks a 50-Million element reduction loop across varying thread counts:

```
# 1. Create working directory
mkdir -p ~/hpc_day02/{scripts,bin,logs}
cd ~/hpc_day02/scripts

# 2. Write the OpenMP vector reduction benchmark (openmp_vector_sum.cpp)
cat << 'EOF' > openmp_vector_sum.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <omp.h>

int main(int argc, char** argv) {
    const size_t N = 50000000; // 50 Million elements
    std::vector<double> data(N, 1.0);
    double total_sum = 0.0;

    int max_threads = omp_get_max_threads();
    std::cout << "============================================================" << std::endl;
    std::cout << "      OPENMP MULTITHREADED VECTOR REDUCTION BENCHMARK     " << std::endl;
    std::cout << "============================================================" << std::endl;
    std::cout << "[*] Problem Size: " << N << " elements ("
              << (N * sizeof(double)) / (1024 * 1024) << " MB memory footprint)" << std::endl;
    std::cout << "[*] Available System Threads: " << max_threads << std::endl;

    // Test scaling across 1, 2, 4, up to max_threads
    for (int t = 1; t <= max_threads; t *= 2) {
        omp_set_num_threads(t);
        total_sum = 0.0;

        auto start = std::chrono::high_resolution_clock::now();

        #pragma omp parallel for reduction(+:total_sum) schedule(static)
        for (size_t i = 0; i < N; ++i) {
            total_sum += data[i];
        }

        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> duration = end - start;

        std::cout << "  • Threads: " << t
                  << " | Latency: " << duration.count() << " ms"
                  << " | Sum: " << (long long)total_sum
                  << " | Status: " << (total_sum == (double)N ? "✅ PASS" : "❌ FAIL")
                  << std::endl;
    }
    std::cout << "============================================================" << std::endl;
    return 0;
}
EOF

# 3. Compile with OpenMP and optimization flags
if command -v icpx >/dev/null 2>&1; then
    icpx -O3 -qopenmp openmp_vector_sum.cpp -o ~/hpc_day02/bin/openmp_sum
else
    g++ -std=c++20 -O3 -fopenmp openmp_vector_sum.cpp -o ~/hpc_day02/bin/openmp_sum
fi

# 4. Execute the binary
~/hpc_day02/bin/openmp_sum
```

🔬 STAGE 2: Data Decomposition & False Sharing Demonstration

Demonstrates the destructive impact of false sharing when multiple threads write to the same 64-byte cache line:

```
cat << 'EOF' > false_sharing_demo.cpp
#include <iostream>
#include <chrono>
#include <omp.h>

constexpr int NUM_THREADS = 4;
constexpr long long ITERATIONS = 100000000;

// Bad struct: variables sit on the SAME 64-byte cache line!
struct BadCounter {
    long long value; // 8 bytes each -> All 4 fit within 32 bytes (1 cache line!)
};

// Good struct: explicitly aligned to 64-byte cache line boundary!
struct alignas(64) GoodCounter {
    long long value;
};

int main() {
    omp_set_num_threads(NUM_THREADS);

    std::cout << "============================================================" << std::endl;
    std::cout << "      FALSE SHARING & CACHE COHERENCY BENCHMARK           " << std::endl;
    std::cout << "============================================================" << std::endl;

    // --- TEST 1: FALSE SHARING (Unpadded) ---
    BadCounter bad[NUM_THREADS] = {{0}, {0}, {0}, {0}};
    auto start1 = std::chrono::high_resolution_clock::now();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITERATIONS; ++i) {
            bad[tid].value++;
        }
    }
    auto end1 = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> time_bad = end1 - start1;
    std::cout << "[1] False Sharing (Same 64-byte Cache Line) : " << time_bad.count() << " ms" << std::endl;

    // --- TEST 2: CACHE ALIGNED (No False Sharing) ---
    GoodCounter good[NUM_THREADS] = {{0}, {0}, {0}, {0}};
    auto start2 = std::chrono::high_resolution_clock::now();

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        for (long long i = 0; i < ITERATIONS; ++i) {
            good[tid].value++;
        }
    }
    auto end2 = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> time_good = end2 - start2;
    std::cout << "[2] Cache Line Aligned (Zero Bus Contention): " << time_good.count() << " ms" << std::endl;

    std::cout << "------------------------------------------------------------" << std::endl;
    std::cout << "[+] Speedup from eliminating False Sharing : "
              << (time_bad.count() / time_good.count()) << "x FASTER!" << std::endl;
    std::cout << "============================================================" << std::endl;
    return 0;
}
EOF
```

```
# Compile and run
g++ -O3 -fopenmp false_sharing_demo.cpp -o ~/hpc_day02/bin/false_sharing
~/hpc_day02/bin/false_sharing
```

🌐 STAGE 3: C++ MPI Distributed Communication

Demonstrates distributed memory execution using standard MPI compiler wrappers:

```
# 1. Install OpenMPI build tools (if not already installed)
sudo apt-get update -qq && sudo apt-get install -y -qq libopenmpi-dev openmpi-bin

# 2. Write the MPI reduction program (mpi_collective_reduction.cpp)
cat << 'EOF' > mpi_collective_reduction.cpp
#include <mpi.h>
#include <iostream>
#include <vector>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank = 0, size = 0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    char hostname[MPI_MAX_PROCESSOR_NAME];
    int len = 0;
    MPI_Get_processor_name(hostname, &len);

    std::cout << "[HPC Node " << hostname << "] Rank " << rank << " of " << size
              << " is computing partial partition." << std::endl;

    long long local_val = (rank + 1) * 100;
    long long global_sum = 0;

    // Collective Reduction: Sum local_val from all ranks into global_sum on Rank 0
    MPI_Reduce(&local_val, &global_sum, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::cout << "\n============================================================" << std::endl;
        std::cout << "[Master Rank 0 Aggregation]: Total Global Sum = " << global_sum << std::endl;
        std::cout << "Successfully reduced across " << size << " distributed ranks." << std::endl;
        std::cout << "============================================================" << std::endl;
    }

    MPI_Finalize();
    return 0;
}
EOF

# 3. Compile and execute across 4 simulated cluster processes
mpicxx -O3 mpi_collective_reduction.cpp -o ~/hpc_day02/bin/mpi_reduction
mpirun --allow-run-as-root --oversubscribe -n 4 ~/hpc_day02/bin/mpi_reduction
```

📊 STAGE 4: Intel AI Analytics Toolkit: Scikit-learn Acceleration Benchmark

Benchmarks unpatched stock Scikit-learn vs. Intel oneDAL `sklearnex` on 500,000 samples with 32 features:

```
cat << 'EOF' > benchmark_kmeans_sklearnex.py
import time
import numpy as np
from sklearn.datasets import make_blobs

print("============================================================")
print("   INTEL oneDAL vs STOCK SCIKIT-LEARN K-MEANS BENCHMARK   ")
print("============================================================")

# 1. Synthesize massive clustering dataset
N_SAMPLES = 500000
N_FEATURES = 32
N_CLUSTERS = 10

print(f"[*] Generating synthetic dataset: {N_SAMPLES:,} samples, {N_FEATURES} features...")
X, _ = make_blobs(n_samples=N_SAMPLES, n_features=N_FEATURES, centers=N_CLUSTERS, random_state=42)
print(f"[+] Dataset memory size: {X.nbytes / (1024 * 1024):.2f} MB\n")

# --- TEST 1: Stock Scikit-learn Run ---
from sklearn.cluster import KMeans as StockKMeans

print("[RUN 1] Executing Standard Stock Scikit-Learn K-Means...")
start_stock = time.time()
kmeans_stock = StockKMeans(n_clusters=N_CLUSTERS, max_iter=20, n_init=1, random_state=42)
kmeans_stock.fit(X)
time_stock = time.time() - start_stock
print(f"  • Stock Scikit-Learn Time: {time_stock:.3f} seconds\n")

# --- TEST 2: Intel oneDAL Accelerated Run ---
print("[RUN 2] Applying Intel oneDAL Acceleration (patch_sklearn)...")
from sklearnex import patch_sklearn
patch_sklearn()

from sklearn.cluster import KMeans as AcceleratedKMeans

start_intel = time.time()
kmeans_intel = AcceleratedKMeans(n_clusters=N_CLUSTERS, max_iter=20, n_init=1, random_state=42)
kmeans_intel.fit(X)
time_intel = time.time() - start_intel
print(f"  • Intel oneDAL Time       : {time_intel:.3f} seconds\n")

# --- RESULTS SUMMARY ---
speedup = time_stock / time_intel if time_intel > 0 else 1.0
print("============================================================")
print(f"  • Stock Scikit-Learn Latency : {time_stock:.3f} s")
print(f"  • Intel oneDAL Latency       : {time_intel:.3f} s")
print(f"  • MEASURED ACCELERATION      : {speedup:.2f}x FASTER!")
print("============================================================")
EOF

python3 benchmark_kmeans_sklearnex.py
```

🌲 STAGE 5: Supervised ML: Random Forest & SVM Acceleration

Tests Random Forest classifier speedup on high-dimensional multi-class classification:

```
cat << 'EOF' > benchmark_random_forest.py
import time
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearnex import unpatch_sklearn, patch_sklearn

print("============================================================")
print("     INTEL oneDAL RANDOM FOREST CLASSIFIER BENCHMARK      ")
print("============================================================")

# Generate classification dataset
X, y = make_classification(n_samples=100000, n_features=30, n_informative=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 1. Unpatched Stock Run
unpatch_sklearn()
from sklearn.ensemble import RandomForestClassifier as StockRF
print("[*] Running Stock Random Forest (100 estimators)...")
t0 = time.time()
rf_stock = StockRF(n_estimators=100, max_depth=12, n_jobs=-1, random_state=42)
rf_stock.fit(X_train, y_train)
t_stock = time.time() - t0
print(f"  • Stock Training Time: {t_stock:.3f} s")

# 2. Intel oneDAL Accelerated Run
patch_sklearn()
from sklearn.ensemble import RandomForestClassifier as IntelRF
print("[*] Running Intel oneDAL Random Forest...")
t0 = time.time()
rf_intel = IntelRF(n_estimators=100, max_depth=12, n_jobs=-1, random_state=42)
rf_intel.fit(X_train, y_train)
t_intel = time.time() - t0
print(f"  • Intel Training Time: {t_intel:.3f} s")

speedup = t_stock / t_intel if t_intel > 0 else 1.0
print("------------------------------------------------------------")
print(f"[+] Random Forest Speedup: {speedup:.2f}x FASTER!")
print("============================================================")
EOF

python3 benchmark_random_forest.py
```

Runbook Status: 100% Complete & Verified | GLA University HPC Masterclass Day 2